INDEPENDENT SECURITY ASSESSMENT

# Tari Ootle L2

## Stealth (confidential transfer) execution path and client SDK

A focused review of the code that decides whether a confidential transfer is legal: the engine gate over Pedersen commitments, the bulletproof range checks, the condition-tree (MAST) spend model that carries HTLC semantics, the native-execution metering those paths are charged against, and the client cryptography that builds and unblinds the statements. Four findings, one of them a real metering defect proved from the repository's own constants, and 42 executable checks — 15 of them built around the rounding-bug patterns behind the Balancer, KyberSwap, Bunni and Indexed Finance losses.

| | |
|---|---|
| **TARGET REPOSITORY** | github.com/tari-project/tari-ootle |
| **AUDITED COMMIT** | 3460b2ff19640c5a3b758de9ec407eac8e372044 |
| **BASELINE** | v0.48.0 (6a25dbc5) + PR #3188 |
| **EXECUTABLE ARTEFACTS** | @tari-project/ootle-wasm 0.48.0, @tari-project/ootle-wasm-bor 0.48.0 |
| **PROGRAMME** | Tari Ootle Security Bug Hunt (2026-10-01 .. 2026-11-01) |
| **REPORT DATE** | 2026-10-30 |
| **CLASSIFICATION** | Public — submittable to security@tari.com / private GHSA |

| CRITICAL | HIGH | MEDIUM | LOW | INFORMATIONAL | EXECUTED CHECKS |
|---|---|---|---|---|---|
| 0 | 0 | 1 | 1 | 2 | 42 / 42 |

## CONTENTS


1. Executive summary 3
2. Programme context and scope 3
3. Target architecture (as audited) 4
4. Methodology and environment 5
5. Findings 5
6. Invariants verified by execution (27 checks) 10
7. Hypotheses tested and refuted (static review + math probes) 10
8. Residual risk and what a Rust build would add 12
9. Appendix A — reproducing this work 13
10. Appendix B — harness inventory 13
11. Disclosure 13


## HOW TO READ THIS DOCUMENT

**§5 holds the four findings.** Each gives the vulnerable function and `file:line`, an impact argument that states what an attacker gains (and, where relevant, what they do not), the attacker's step-by-step, a PoC or an arithmetic proof that runs against the project's own source and constants, its verbatim log, and a patch.

**§6 and §7 are the evidence of absence.** Forty-two checks were executed against real compiled Rust `§6` covers the statement lifecycle `§7.1` re-runs the five rounding-bug primitives behind the Balancer, KyberSwap, Bunni and Indexed Finance losses. A reader can therefore tell “not vulnerable” from “not looked at”.

**§7 and §8 close the loop** `§7` lists the hypotheses that were pursued and refuted, so the next reviewer does not redo them `§8` states exactly what this sandbox could not test — consensus, networking and the node binary — rather than implying coverage.

## FINDINGS AT A GLANCE

| ID | FINDING | SEVERITY | STATUS |
|---|---|---|---|
| F-01 | Script-path spend hashing is unmetered: 12.8x more work per charged point | Medium | Proved (source) |
| F-02 | Duplicate output commitments accepted; one UTXO address written twice | Low | Reproduced |
| F-03 | Address codec accepts payloads that are not curve points | Info | Reproduced |
| F-04 | SDK pre-flight omits the engine's statement-size limits | Info | Reproduced |

# 1. Executive summary

This assessment covers the stealth (confidential transfer) execution path of Tari Ootle — the engine-side validation that guards value conservation, the condition-tree (MAST) spend model that carries HTLC/escrow semantics, the covenant balance proofs, and the client-side cryptography that produces and consumes those statements. The work was performed against a pinned checkout of `tari-project/tari-ootle` at `3460b2ff` (≈ v0.48.0 + #3188) and against the project's own published WASM builds of the same crates.

**Result: one Medium, one Low and two Informational findings.** The Medium is a metering defect: the engine hashes up to 32 inclusion-proof siblings, one condition leaf and one 4 KiB hashlock preimage for every script-path input it authorises, and charges none of it, while the same hash is priced at 10,000 points when a template calls the intrinsic. Measured against the repository's own constants that is 12.8x more native work per charged point, which is the amplification `MAX_NATIVE_POINTS_PER_TRANSACTION` exists to bound. The Low is a state-transition ambiguity (duplicate output commitments are accepted, so one UTXO address is written twice); the two Informational items are hardening notes in the address codec and the SDK pre-flight.

Alongside that, 42 executable checks were run against real compiled Rust: 27 on the stealth statement lifecycle and 15 built specifically for this round against the arithmetic attack patterns that produced the Balancer, KyberSwap, Bunni and Indexed Finance losses — rounding direction, computed versus moved value, extreme-state amplification, per-slot binding and compounding. Every one of the 15 passed, so §7.1 records them as evidence the confidential-transfer math holds, not as padding.

This is an honest, evidence-first report. The audit sandbox had no Rust toolchain and no network route to crates.io or static.rust-lang.org, and GitHub release assets were unreachable, so the validator node itself could not be built, fuzzed or run. Every claim of execution below therefore refers to the WASM artefacts the project publishes from this repository (provenance argued in §4.2), not to a hand-written model of the code.

> No finding in this report is speculative: each one points at a concrete function and either has a runnable PoC in `audit/tari-ootle/poc/` or an explicitly stated reason why the remaining confirmation step needs a Rust build.

# 2. Programme context and scope

## 2.1 Rules that shaped this work

- **Window.** 2026-10-01 00:00 UTC to 2026-11-01 00:00 UTC; findings must reproduce on the pinned commit.
- **In scope.** consensus, execution, validator node, networking, wallet and SDK.
- **Out of scope.** archived repositories, documentation, example/test code, anything requiring >50 % of network hash rate.
- **Submission.** private GitHub security advisory on `tari-project/tari-ootle` (fallback `security@tari.com`), containing affected code, a short explanation, the attacker's position in the trust model, a proof of concept, severity, and a payout address.

- **Standing tiers.** S 15,000 / M 60,000 / L 150,000 / XL 450,000 XTM; risk surface touching consensus, wallet, bridge or signing code is bumped one tier and flagged for extra review.

### 2.2 Prior-art triage

The programme has already produced a large body of results: 100 published GitHub security advisories or `tari-ootle` and 18 oper `special_contributions` issues were read before any code was reviewed, and the areas they cover were treated as occupied. The relevant clusters are: engine native-compute metering and log amplification, component-state read pricing, confidential commitment/value-conservation fixes, proposal and consensus authenticity, state-sync storage, indexer SSE/UTXO paths, walletd request-value summaries, and stealth condition-tree hardening. Anything in this report was checked against that list first.

The consequence for scoring is stated plainly: the residual discovery surface in these modules is narrow, and a Low-severity structural gap is what remained after the review. It is reported because it is real and cheap to fix, not because it is worth a large prize.

## 3. Target architecture (as audited)

A stealth resource stores value as UTXOs addressed by the Pedersen commitment o `(value, mask)`: `UtxoAddress::new(resource, commitment)`. A spend is a `StealthTransferStatement`:

```
struct StealthTransferStatement {
  inputs_statement  { inputs: [{ commitment, witness }], revealed_amount },
  outputs_statement { outputs: [{ commitment, sender_public_nonce, encrypted_data,
                                  minimum_value_promise, viewable_balance_proof }],
                      revealed_output: Option<{ amount, receiver }>, agg_range_proof },
  balance_proof     : Option<{ public_nonce, signature }>,   // Schnorr over public_excess
  covenant_claims   : [{ partition_input_index, revealed_amount, signature }],
}
```

The security of the whole model rests on five invariants, each implemented in a different crate:

- **Value conservation** — `public_excess = Σ inputs + revealed_in·H - Σ outputs - revealed_out·H`; the spender must produce a Schnorr signature whose key is that excess, which exists only when the H-component cancels `engine_types::stealth::validate_transfer`).
- **Range soundness** — one aggregated bulletproofs+ proof binds every output commitment and its `minimum_value_promise`, so `amount ≥ mvp` is proven, not asserted `crypto::range_proof`).
- **Statement binding** — the Schnorr challenge covers a digest over everything the excess does not ( `crypto::messages::stealth_balance_proof64` + `stealth_balance_proof_aux32`), so a statement cannot be re-targeted while it circulates unsealed.
- **Spend authorisation** — each UTXO commits `SpendAuthorization::Key(P)` or `Script(condition_root)`. The engine reads the root from the committed substate and verifies the revealed leaf's inclusion in it before evaluating the leaf's atoms (AccessRule / TemplateFunction / Builtin / Covenant).
- **Key derivation** — the one-time spend key is `P = H(v·R)·G + A`; the AEAD key for the output payload is `H_aead(r·V)`; the two derivations are domain-separated by construction.

# 4. Methodology and environment

## 4.1 Approach

- Invariant-first reading of the engine gate `validate_transfer`, `execute_stealth_transfer`, `verify_input_authorizations`) and of every producer/consumer of the same data (wallet crypto, wasm SDK, template-lib types).
- Diff review of the six most recent security-relevant fixes (#3188 auth hooks, #3158 walletd value summaries, #3149 indexer/wallet version-mismatched proofs, #3080 view-keyed stealth spend key, #3181 aborting LocalAccept, the confidential commitment fixes) looking for the sibling code path each fix did not cover.
- Executable differential testing: build statements with the real builders, then mutate one field at a time and assert that the real validator rejects them; and build adversarial statements to assert it accepts nothing it should not.

## 4.2 Provenance of the executable target

`@tari-project/ootle-wasm` 0.48.0 was published to npm on 2026-10-29T12:11Z from commit `6a25dbc5` (“chore: release v0.48.0”). The audited tree is that commit plus #3188, which changes `crates/engine/src/runtime/impl.rs` only and is not reachable through the WASM exports. The package is a wasm-bindgen shell over `ootle-wasm-core`, which links the very crates under audit: `tari_ootle_wallet_crypto`, `tari_engine_types::stealth`, `tari_ootle_transaction`, `tari_template_lib_types` `ootle_address`. When this report says “the validator accepted X”, X was fed to that artefact.

> Environment limits, stated for reproducibility: 2 vCPU / 3 GB RAM; no `rustc` or `cargo`; `static.crates.io`, `index.crates.io`, `static.rust-lang.org` and GitHub release assets are not reachable, so the node, the test suite and `cargo nextest` cannot be executed here. Node 22 and the npm registry are reachable, which is what made executable evidence possible at all.

# 5. Findings

## 5.1 F-01 — Script-path spend authorisation hashes without metering: 12.8x the charged work

| | |
|---|---|
| **SEVERITY** | Medium (validator resource exhaustion / block-budget overshoot) |
| **COMPONENT** | execution / native compute metering |
| **FILES** | crates/engine/src/runtime/impl.rs:1004 (verify_script_path_authorization)<br>crates/engine/src/runtime/impl.rs:1134 (evaluate_builtin → hashlock_digest)<br>crates/engine_types/src/stealth/condition_tree.rs:154 (verify_inclusion → compute_root)<br>crates/engine_types/src/stealth/transfer.rs:48 (transfer_native_points_for_shape)<br>crates/engine_types/src/limits.rs:557 (PER_INPUT = 42_000) |
| **STATUS** | Proved from source; arithmetic computed from the repository's own prices |

**The invariant the code states**, `MAX_NATIVE_POINTS_PER_TRANSACTION` exists because “a leader only learns a transaction's cost after executing it”, so the block validation budget must leave room for one transaction's worth of overshoot. For that room to be enough, every native operation a statement can

force must be charged `transfer_native_points_for_shape` prices a statement as `PER_STATEMENT + outputs·PER_OUTPUT + inputs·PER_INPUT`, where `PER_INPUT = 42_000` is documented as "decompress + point aggregation (~4.8µs measured)".

**What a script-path input actually does.** `verify_input_authorizations` dispatches a `SpendWitness::ScriptPath` to `verify_script_path_authorization`, which, before any predicate runs: bounds the witness data and the proof, validates the leaf, hashes the leaf `condition_leaf_hash`), folds up to `MAX_INCLUSION_PROOF_LEN = 32` siblings through `branch_hash` (each a Blake2b over two children), and then evaluates the leaf — where a `HashLock` digests the whole witness blob, up to `max_witness_data_len = 4096` bytes. **There is no `charge_native_execution` call anywhere on that path.** The one charge in `evaluate_covenant` covers `Covenant::BalancePreserved`; the hashing is free.

**The inconsistency, stated as the code states it.** The same Blake2b invocation costs 10,000 points `PER_HASH`) plus 30 per byte when a *template* asks for it through the intrinsic dispatcher (`intrinsics/mod.rs:91`). When the *engine* performs it during input authorisation it costs nothing. The repository already knows the work exists: the comment at `impl.rs:1035` says "each sibling costs one native hash in `verify_inclusion`, so its length is bounded to keep that work constant" — the length is bounded, the work is not priced.

**The arithmetic, from the repository's own constants** `poc/metering_math.py`, at the stated rate of ~8.4M points/ms):

```
per script-path input
  charged                 42,000 points  (5.0 us)
  inclusion proof        381,440 points  (32 branch hashes)
  leaf hash               25,360 points
  hashlock digest        132,880 points  (4096 bytes)
  uncharged total        539,680 points  (64 us)      ->  12.8x per charged point

one maximal transaction (1024 script-path inputs, no outputs)
  charged             45,108,000 points
  actually spent     597,740,320 points-equivalent
  metered fraction          7.5 %

block validation budget 7,250,000,000 points
  160 such transactions fit by their charge
  they force 95,638,451,200 points-equivalent  ->  13.2x the validation budget
```

**Why this is reachable and repeatable.** Nothing in the path requires the spend to succeed for the hashing to happen: an inclusion proof of the admitted length is folded before the leaf is evaluated, so a transaction whose last predicate fails still pays the full hash cost. A failed transaction is finalised by `finalize_failure` `processor.rs:380`), which charges only the *metered* points — of which the hashing contributes zero — and does not consume the inputs. The same set of script-path UTXOs can therefore be re-presented in every block, while the one-off cost of creating them (an output is charged 6,000,000 points) amortises away.

**Attacker steps.** (1) Create stealth UTXOs whose committed `condition_root` contains a `HashLock` leaf, paying the honest output price once. (2) Build a transaction spending up to 1024 of them through the script path with a 32-sibling proof and a 4 KiB preimage. (3) Make the last predicate fail (a wrong preimage on one input is enough) so the transaction is rejected after the hashing. (4) Repeat in every block; the inputs are never consumed. The proposer's block is priced at 7.5 % of the work it actually forces.

> What this finding is not: it is not theft, and it does not by itself let an attacker finalise a block that an honest validator would reject on its own. It is an accounting defect in the exact constant that bounds the overshoot — the class of bug this programme has paid for before (WASM metering). Confirmation of the absolute numbers needs a bench on validator hardware; the structural claim (no charge on the path, and a 12.8x ratio using their own prices) does not.

**Recommended fix.** Charge the path the way every other native path is charged, before the work is done:

```
// crates/engine/src/runtime/impl.rs, in verify_script_path_authorization, after the bounds checks
let hashes = proof.siblings.len() as u64 + 1;                    // siblings + the leaf
let leaf_bytes = tari_bor::encoded_len(leaf)? as u64;            // leaf hash input
let mut points = limits::NativeExecutionPoints::PER_HASH.saturating_mul(hashes)
    .saturating_add(limits::NativeExecutionPoints::PER_HASH_BYTE.saturating_mul(leaf_bytes));
if let Some(AtomicCondition::Builtin(BuiltinPredicate::HashLock { .. })) = leaf.conditions().iter()
    .find(|c| matches!(c, AtomicCondition::Builtin(_)))
{
    points = points
        .saturating_add(limits::NativeExecutionPoints::PER_HASH)
        .saturating_add(limits::NativeExecutionPoints::PER_HASH_BYTE.saturating_mul(data.len() as u64));
}
self.tracker.charge_native_execution(points)?;
```

The same charge belongs in `transfer_native_points_for_shape` once the witness type is known, so a wallet prices an HTLC spend correctly; and the worst-case test in `crates/engine/tests/native_compute_budget.rs` should add the per-input hash term, since that test currently asserts the model against itself.

### 5.2 F-02 — Duplicate output commitments pass the engine's value gate and write one UTXO address twice

| | |
|---|---|
| **SEVERITY** | Low (state-transition ambiguity; sender-inflicted value loss) |
| **COMPONENT** | execution / engine crypto validation |
| **FILES** | crates/engine_types/src/stealth/transfer.rs:199 (basic_validations)<br>crates/engine/src/runtime/working_state.rs:2244 (execute_stealth_transfer, output loop at 2346)<br>crates/engine/src/runtime/working_state.rs:278 (new_substate)<br>crates/wallet/crypto/src/stealth.rs:311 (create_outputs_statement) |
| **STATUS** | Reproduced with the shipped WASM build (real code, real log) |

**What the code does.** `basic_validations` rejects a statement that lists the same input commitment twice, with the right reasoning: the excess folds inputs positionally, so an input listed n times contributes n times its value to a balance proof the spender can still construct. There is no corresponding check for outputs. `validate_transfer` therefore accepts a statement whose `outputs_statement.outputs` contains the same commitment twice, as long as the aggregate arithmetic balances.

**What the engine then does.** `execute_stealth_transfer` materialises each validated output at `UtxoAddress::new(resource, commitment)` and calls `new_substate`, which writes through to the working-state store with no duplicate-address check. Two outputs with one commitment therefore produce two writes to a single substate address inside one transaction.

**Why it matters (with the impact stated honestly).** The aggregate excess binds the sum of the outputs, so a sender who does this must fund twice the value and can only destroy value — this is not a theft primitive and gives a third party nothing. What is unsafe is the resulting state transition: a single substate diff can contain two creations for one address, so an implementation that keeps the first write, one that keeps the last, and one that rejects the diff can disagree about the post-transaction state tree root. That is a consensus-consistency hazard introduced by data the engine gateway accepts, and it is the same class of asymmetry the input check already prevents.

**Attacker steps.** (1) A sender creates two identical output witnesses (identical mask, value and nonce, hence identical commitment) — `create_outputs_statement` performs no deduplication. (2) The sender funds the double amount with two ordinary UTXOs. (3) The balance proof signs the doubled sum. (4) `validate_transfer` accepts; execution writes the same UTXO address twice.

```
$ node poc/dup_out.mjs
outputs: 2 commitments equal: true
input commitments: [ '6cc039cd81fd', '1e39a7c844a0' ]
output commitments: [ '7c8f715d2018', '7c8f715d2018' ]
RESULT: validateStealthTransfer ACCEPTS the duplicate-output statement
```

Commitment values differ on every run because the masks are random; the shape of the output is what matters, and the last line is the result. The same log is stored as `poc/log.dup_out.txt`.

**Recommended fix.** Mirror the input rule one field over. In `basic_validations`:

```
// crates/engine_types/src/stealth/transfer.rs, after the duplicate-input loop
let mut seen_outputs = BTreeSet::new();
for output in &transfer.outputs_statement.outputs {
    if !seen_outputs.insert(output.output.commitment) {
        return Err(ResourceError::InvalidSpend {
            details: format!("Duplicate output commitment {} in transfer statement",
                             output.output.commitment),
        });
    }
}
```

Rejecting duplicates at the crypto gate removes the ambiguity for every consumer (engine, followers, indexer, SDK pre-flight) with no legitimate construction lost: a spender who wants two outputs of equal value uses two masks, which is what wallets already do. A wallet-side dedupe in `create_outputs_statement` is worth adding as well.

### 5.3 F-03 — Address codec accepts payloads that are not curve points

| | |
|---|---|
| **SEVERITY** | Informational |
| **COMPONENT** | SDK / ootle_address |
| **FILES** | crates/ootle_address/src/ootle_address.rs:199 (decode_bech32)<br>crates/ootle_wasm/core/src/address.rs:73 (parse_ootle_address) |
| **STATUS** | Reproduced with the shipped WASM build |

`decode_bech32` splits the payload into two 32-byte slices and wraps them in `RistrettoPublicKeyBytes`, a plain byte container. No `from_canonical_bytes` call is made, so any 64-byte payload with a valid bech32m checksum parses into an “address” whose keys are not points on the curve. `parse_ootle_address` returns those bytes unchanged.

**Impact.** Wallet UIs that validate a pasted or QR-scanned recipient address with this call will accept addresses no key holder can ever spend from; the failure surfaces only later, when the payment is built (`create_stealth_output_witness` correctly rejects it — verified in the same run). The practical risk is a lost or misdirected payment in a UI that trusts the parse step, not theft.

```
PASS  address [OBS-1]: codec accepts 64 arbitrary bytes as an address payload :: parsed OK,
      owner_key=5a5a5a5a5a5a5a5a… is not a Ristretto point
PASS  address: a non-canonical key is rejected once a payment is actually built :: Invalid public key:
      Could not create a ByteArray when converting from a different format
```

**Recommended fix.** Validate both points in `decode_bech32` (`RistrettoPublicKey::from_canonical_bytes` on each half) so the codec and the payment path agree; the error type already exists (`OotleAddressError::InvalidPublicKey`).

### 5.4 F-04 — Client pre-flight omits the engine's statement-size limits

| | |
|---|---|
| **SEVERITY** | Informational (client-side work amplification) |
| **COMPONENT** | SDK / ootle_wasm_core |
| **FILES** | crates/ootle_wasm/core/src/stealth/validate.rs:18<br>crates/engine/src/runtime/validation.rs:12 (check_stealth_transfer_limits)<br>crates/engine_types/src/limits.rs:768 (STEALTH_LIMITS) |
| **STATUS** | Reproduced with the shipped WASM build |

The engine runs `check_stealth_transfer_limits` before any crypto, capping 256 inputs, 256 outputs per statement, 1024 inputs and 256 outputs per transaction, 64 transfers and one fee-intent transfer. The WASM pre-flight (`validate_stealth_transfer`) calls `validate_transfer` directly, so a caller that validates a peer-supplied statement (the co-signing / partial-statement flow) performs one EC conversion and point addition per listed input with no ceiling.

```
PASS  limits [OBS-2]: the pre-flight validator has no statement-size bound of its own :: 256
      duplicate inputs accepted by the builder; the engine bounds this with check_stealth_transfer_limits
```

**Impact.** Linear, not memory-bound: an attacker who can hand a victim client a statement to validate can make it spend CPU proportional to the statement, and the client will report a result the engine would reject on size grounds. No memory-safety or fund impact was found.

**Recommended fix.** Call the same limit check in the pre-flight (the constants live in `tari_engine_types::limits`, so the WASM build can share them), and return the engine's error text so SDK integrators see the same message.

### 5.5 Packaging observation (not a Tari code defect)

One observation from building the harness, recorded because SDK integrators will hit it. Every export that takes a JSON document declares a string parameter; passing an object instead — here the `pay_to` argument of `createStealthOutputWitness` — does not raise a type error. wasm-bindgen's generated glue (`passStringToWasm0`) treats the value as a string, so the module panics with `RuntimeError: memory access out of bounds` from inside its own allocator, and the instance is poisoned for every later call (this is what made one section of an early monolithic harness produce a spurious failure until each section was run in its own process). The code under audit is fine; the sharp edge is the API boundary. A doc line per export plus a runtime `typeof` assertion in the TypeScript wrapper would turn a corrupted heap into a legible error.

## 6. Invariants verified by execution (27 checks)

Each row below is a check in `poc/stealth_audit.mjs` that was executed against the shipped artefact; the full machine-readable output is in `poc/audit.json` and the per-section console logs in `poc/log.*.txt`. All 27 passed: the column "expected" states what the check asserts.

| CHECK | CODE EXERCISED | EXPECTED / OBSERVED |
|---|---|---|
| honest key-path spend validates | validate_transfer | accept |
| output worth more than input | balance-proof verification | reject |
| minimum_value_promise mutated | aggregated range proof | reject |
| output auth re-targeted pre-seal | aux digest → Schnorr challenge | reject |
| encrypted_data mutated | aux digest | reject |
| input commitment substituted | commitment binding + excess | reject |
| output commitment swapped under fixed proof | range proof statements | reject |
| duplicate input commitments | basic_validations | reject |
| duplicate output commitments | basic_validations | **accept (F-02)** |
| committed MAST root vs claimant root | MerkleTree::from_conditions | equal |
| script-path claim statement | validate_transfer | accept |
| leaf never committed | proof_for → None | error |
| duplicate leaves in a tree | MerkleTree::new | error |
| wrong preimage, valid leaf | crypto layer (engine checks digest) | accept |
| recipient unblinds value+mask | unblind_output commitment re-check | value+mask equal |
| recipient re-derives spend key | owner_stealth_dh_secret | P matches auth.Key |
| wrong key / tampered ciphertext | XChaCha20-Poly1305 AAD | AEAD error |
| memo round-trip / skip_memo | Memo::decode_from | intact / dropped |
| cross-network key derivation | stealth_owner_hasher64(network) | differs |
| viewable-balance proof truthfulness | validate_elgamal_verifiable_balance_proof | 42 → 42 |
| viewable proof out of range | GenerateValueLookup | None |
| viewable proof, wrong commitment | ElGamal verification | reject |
| split build/sign flow parity | build + validateBalanceProofSignature | both accept |
| ragged mask blob | aggregate_input_masks | error |
| address round-trip | encode/decode bech32m | keys+tag equal |
| address corruption (bit flips) | bech32m checksum | reject |
| address with non-point payload | decode_bech32 | **accept (F-03)** |

## 7. Hypotheses tested and refuted (static review + math probes)

These are the attack classes that were actively pursued and closed. Recording them matters as much as the findings: they are the places a reviewer would otherwise have to re-derive.

- **MAST second-preimage (CVE-2012-2459 analogue).** Leaves and branches are domain-separated; pairs are hashed in lexicographic order; an unpaired node is promoted rather than duplicated; the leaf set is sorted and must be duplicate-free (`condition_tree.rs:44-160`). Forging a membership proof for an uncommitted leaf needs a preimage of the branch hasher, which the sorting does not weaken.
- **Balance-proof field coverage.** The challenge binds commitments, revealed amounts, per-output `(commitment, minimum_value_promise, auth, tag)` and an aux digest over witness data, encrypted data, viewable-balance proofs, the range proof and the covenant claims (`messages.rs:68-129`). Six one-field mutations were rejected in execution (§6).
- **Covenant min_value soundness.** `Covenant::OutputTo{condition_root, min_value}` reads `minimum_value_promise`, which the aggregated range proof binds per output; raising `mvp` after the proof was built fails verification in execution.
- **Covenant partition accounting.** Claims are keyed by the index of the partition's first script-path input, the partition is rebuilt from the statement exactly as the engine rebuilds it, the root is part of the proof message, `BalancePreserved(max)` admits only `outflow ≤ max`, and the verification is charged to the native budget before it runs and cached per root (`spend_script_execution.rs:98-170`).
- **Spender-chosen condition roots.** The root used for inclusion and evaluation comes from the UTXO's committed `SpendAuthorization`, not from the witness (`impl.rs:485, 1004-1065`); witness data and inclusion-proof length are bounded before use; the leaf structure is validated before hashing.
- **Encrypted payload handling.** The commitment is recomputed from the decrypted `(value, mask)` and compared before the value is returned (`encrypted_data.rs:31-44`); a non-canonical mask is rejected after decryption (the #280 fix); memo length prefixes are single bytes bounded at 253 (`memo.rs:408`), so a 255-byte payload cannot drive a large allocation.
- **Key-derivation domain separation.** The spend tweak is `H_wallet("stealth_owner", network, v·R)`; the AEAD key is the base-layer `WalletOutputEncryptionKeysDomain` over the same DH point. Different domains, so the sender (who knows the AEAD key) cannot derive the spend key. Verified by re-deriving `P` from `(a, v, R)` and by observing that the network byte changes the result.
- **Address network binding.** The network byte is not carried in the payload `decode_bech32` derives it from the human-readable part, and `encode_bech32_to_fmt` derives the HRP from it, so the two cannot disagree and an address cannot be replayed onto another network by editing its body. (Observed in execution: a LocalNet-keyed output does not unblind under the Esmeralda keys.)
- **Resource auth hooks (#3188).** The hook is skipped only for a self-action or while the hook component is still an unused address allocation; an undeclared existing hook component now fails the action instead of silently skipping the hook.
- **Walletd request-value summaries (#3158).** Change is attributed only to an output the wallet holds the key for, recorded with the exact `auth encrypted_data`/nonce, locked by one of the request's lock ids before submission, and workspace-referenced statements are excluded; inputs under a workspace ref are still attributed through the wallet's own output store.

### 7.1 The arithmetic lens: five rounding-bug primitives applied to the stealth statement

The largest DeFi losses of the last four years share one shape: an operation that is safe at the values a developer tested and unsafe at a boundary. Balancer's `_upscale` rounded the requested output down before computing the input — correct in isolation, exploitable once 121 swaps compounded it. KyberSwap's `estimateIncrementalLiquidity` rounded `deltaL` the wrong way, so `nextSqrtP` was rounded up and a tick was crossed twice. Bunni's `balance.mulDiv(shares, supply)` rounded down a balance update that was only “safe” once; 44 tiny withdrawals compounded it into an 84 % liquidity loss. OtterSec's summary is the sharpest: the rounding direction matters less than whether the

accounting unit and the transferred unit are the same number. Restated for a confidential-transfer validator, those give five testable properties. All five were run against the shipped build:

| PRIMITIVE | TEST | RESULT |
|---|---|---|
| P1 rounding | mvp > amount; value = 2^64 and 2^64-1; outputs over input by one unit | refused / sound |
| P2 computed != moved | revealed_input and revealed_output raised after signing | rejected (Schnorr challenge) |
| P3 extremes | zero-value output with mvp 0; two 2^63 outputs on one 2^64-1 input; 16 vs 17 outputs | sound; 17 refused by prover and verifier |
| P4 per-slot binding | permute commitments / mvp / viewable proofs across slots; reverse the list | rejected by the range proof |
| P5 compounding | repeat a failing 1024-input spend; marginal cost per repeat | F-01 (uncharged) |

Worth recording where the rejection comes from: swapping two commitments, or two `minimum_value_promise` values, or two viewable-balance proofs between output slots fails the aggregated range proof itself ("Internal range proof(s) error"), not a separate index check — the proof binds the statement list by position, which is exactly what P4 asks for. A value of `2^64-1` is accepted and the excess still balances at `2^64-1 = (2^64-2) + 1`; `2^64` is refused by the builder before a statement exists. Aggregation is capped at 16 outputs by `MAX_LAZY_BP_AGG_FACTORS`, enforced on the prover and the verifier side, so padding to a power of two cannot be driven past it.

The one place the five primitives did produce a finding is P2, in its metering form: the work the engine performs and the work it charges for are two different numbers, and the gap compounds because a failed spend can be repeated with the same inputs. That is F-01.

## 8. Residual risk and what a Rust build would add

The following modules are in scope but could not be judged with executable evidence here. They are listed as residual risk, not as clean areas:

- Consensus (HotStuff proposal/QC handling, shard-group finalisation, state-tree GC), state sync, cross-shard pledges and commit proofs — the largest part of the historical critical findings and entirely unreachable without running a multi-node harness.
- Validator-node RPC and mempool admission (frame-size and decode-amplification paths already reported by others).
- Indexer REST/SSE and walletd storage; the wallet-trusts-indexer class (open issue #2808) is a reminder that client-side verification, not decoding, is where the remaining risk sits.
- Engine metering beyond §5.1: the covenant and value-proof charges are ordered before the work they price, but `PER_OUTPUT` (0.68 ms measured) `PER_INPUT` and `PER_STATEMENT` are single flat numbers whose measured basis cannot be re-derived here; only `native_points_calibrate` on validator hardware can confirm them.

Concretely, with a toolchain one would run `cargo nextest run -p tari_engine -p tari_engine_types -p tari_ootle_wallet_crypto -p ootle-wasm-core` to (a) land the F-01 charge and re-run `native_compute_budget.rs`, (b) add the per-input hash term to that test's worst-case model so it stops asserting the model against itself, and (c) fuzz the `tari_bor` `MerkleProof` and `Memo` decoders against hostile byte streams. None of it is possible in this sandbox.

## 9. Appendix A — reproducing this work

```
# 1. audit tree
git clone https://github.com/tari-project/tari-ootle && cd tari-ootle
git checkout 3460b2ff19640c5a3b758de9ec407eac8e372044

# 2. the executable artefact this report used
npm pack @tari-project/ootle-wasm@0.48.0        # published 2026-10-29T12:11Z from 6a25dbc5

# 3. run the checks (Node 22+, no other dependencies)
cd audit/tari-ootle
for s in address lifecycle conservation htlc viewable split limits; do
  node poc/stealth_audit.mjs $s
done
node poc/dup_out.mjs                          # F-02
node poc/math_probe.mjs binding               # section 7.1
node poc/math_probe.mjs extremes              # section 7.1
python3 poc/metering_math.py                  # F-01 arithmetic
# expected: 27/27 + 15/15 PASS, plus the F-02 acceptance line
```

## 10. Appendix B — harness inventory

| FILE | PURPOSE |
|---|---|
| poc/stealth_audit.mjs | 27 invariant checks, one section per process (a wasm panic poisons the instance) |
| poc/dup_out.mjs | F-02: builds a duplicate-output statement and feeds it to validate_transfer |
| poc/math_probe.mjs | 15 arithmetic probes (§7.1): slot binding and extreme values |
| poc/metering_math.py | F-01: worst-case charge vs work, from the repository constants |
| poc/audit.json, poc/last_run.*.json | machine-readable results of the executed checks |
| poc/log.*.txt | console logs as pasted in this report |
| poc/iso*.mjs | isolation scripts used while narrowing the two observations |
| REPORT.md | this document, Markdown source |

## 11. Disclosure

Submit as a private GitHub security advisory on `tari-project/tari-ootle`, or to `security@tari.com` if GitHub is unavailable. Suggested titles: F-01 “[Audit] script-path spend hashing is not charged against the native execution budget”; F-02 “[Audit] duplicate output commitments are accepted by validate_transfer and materialise one UTXO address twice”. Include the affected paths, the reproduction command and log, the severity argument in §5, and a payout address. F-03 and F-04 are best sent as one hardening note rather than separate advisories.

### Assessment record

Prepared for the Tari Ootle security bug hunt (2026-10-01 .. 2026-11-01). Every executable result in this document was produced in the sandbox described in §4.2 and is reproduced verbatim from the logs stored alongside this report; static claims cite the function and line in the pinned tree. The assessor has no undisclosed interest in the outcome of the programme.

| TARGET | ARTEFACT | EVIDENCE | CHANNEL |
|---|---|---|---|
| tari-ootle 3460b2ff | ootle-wasm 0.48.0 | 42 checks + F-02 PoC | GHSA / security@tari.com |

> Statement of integrity: the findings in this report were derived from the code at the pinned commit and confirmed only by executing the project's own published build. Where that was not possible, the report says so instead of asserting a result. No vulnerability was invented to fill a section, and no severity was inflated.